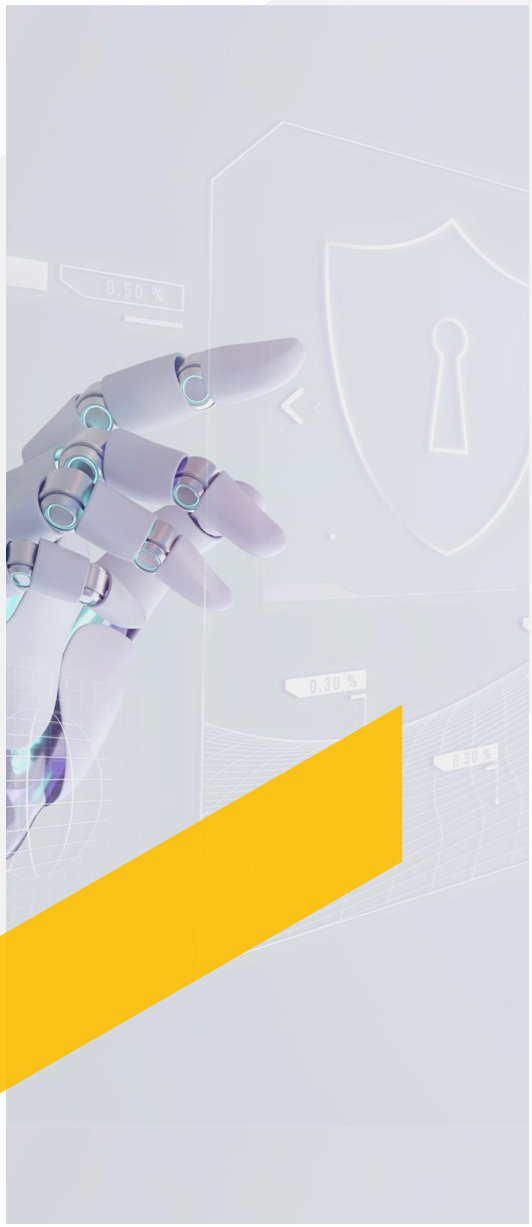
CYBERNERDS SOLUTIONS

Application Security Assessment Report

Nov 2024

CYBERNERDS SOLUTIONS PVT. LTD.
A CERT-IN Empaneled Security Auditor



Disclaimer

Whilst all due care and diligence have been taken in the preparation of this document it is not impossible that a document of this nature may contain errors or omissions as a result of a misunderstanding of Clients' requirements. In particular, any recommendations are made in good faith as guidelines to assist the client in the evaluation and must not be constructed as warranties of any kind. Findings in this report are based on various tests conducted using third-party tools and CYBERNERDS has put its best efforts to eliminate all the false positives reported by these tools.

All terms mentioned in this document that are known to be trademarks or service marks have been appropriately capitalized. CYBERNERDS cannot attest to the accuracy of this information. Use of a term in this document should not be regarded as affecting the validity of any trademark or service mark.

This document has been prepared by **CYBERNERDS's** Cyber Security team for the consideration **Tamil Nadu Geographical Information System (TNGIS) - TNeGA.**

Table of Contents

1. Project Details

Title	Web Application Security Assessment		
Version	1.0		
Document Id	CNS/R/2024/11/WI001		
Hash Value	bb0edb1a65995badc99df248f8d391b392b0a4656174efa28aec29e9f6b60f60e1d68 bd542a21715ebe228ad7c1474c326bc05512e6e38d904e32c7542436375		
Client Name	Tamil Nadu Geographical Information System (TNGIS) - TNeGA		
Project Name	Web Application Security Assessment for Unified State Scholarship Portal		
Assessment Type	External	Grey Box	
Tester	Danavarsini	Security Engineer	
Reviewer	Siva	Principal Security Consultant	
Test Start	14 Oct 2024	**Test End**	4 Nov 2024

2. Scope

Tamil Nadu Geographical Information System (TNGIS) - TNeGA has provided the following application as scope.

S/N	Scope
1.	https://tngis.tnega.org/ssp_frontend/

3. Executive Summary

CYBERNERDS has performed a security assessment of the application above mentioned in scope. The objective was to test the security posture of the application.

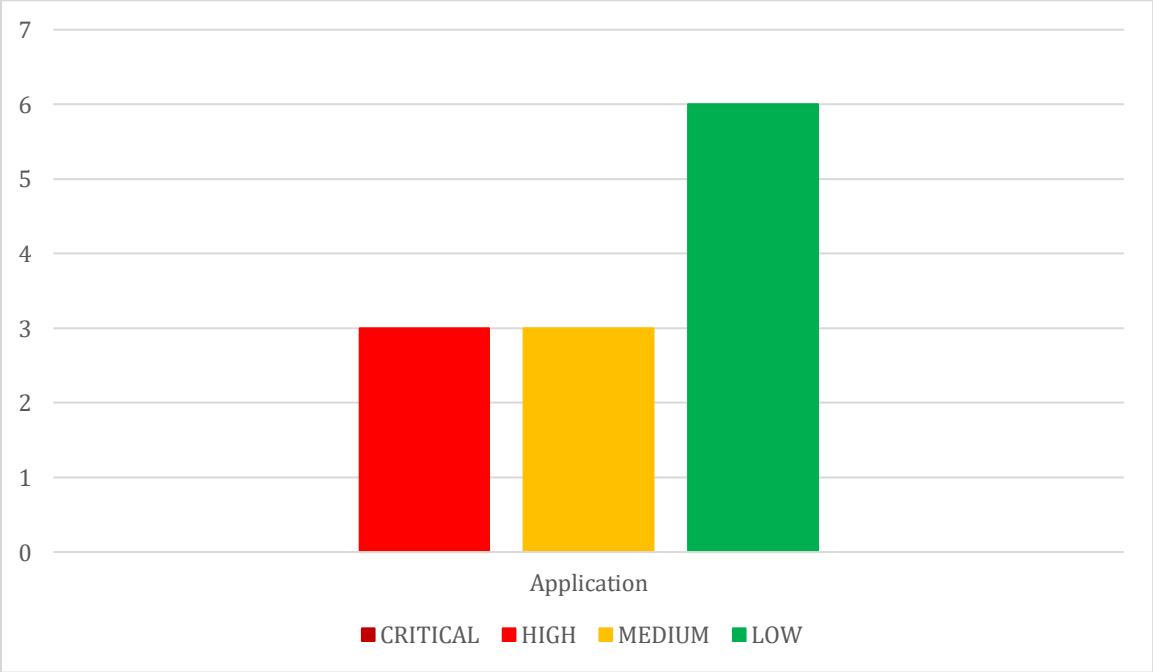
In this test, CYBERNERDS has found **0 Critical, 3 High**, **3 Medium, 6 Low** and **5 Info** risk vulnerabilities.

Vulnerability statistics: Host-wise

S.No	***URL***	***Risk Level***			
		CRITICAL	**HIGH**	**MEDIUM**	**LOW**
1.	https://tngis.tnega.org/ssp_frontend/	0	3	3	6
Total No of Vulnerabilities		**12**			

Number of Vulnerabilities and Risks

* This below graph represents no. of critical, high, medium & low-risk vulnerabilities in the application.



4. Technical Summary

OWASP has rated the Top Ten Vulnerabilities found in applications worldwide.

The table shows how the application compares with respect to the OWASP Top 10 list.

CYBERNERDS's application security tests are modelled along the methodologies specified by the Open Web Applications Security Project (OWASP).

S/N	*Vulnerabilities (OWASP TOP 10-2021)*	*Test Status*
A1	Broken Access Control	Unsafe
A2	Cryptographic Failures	Safe
A3	Injection	Unsafe
A4	Insecure Design	Unsafe
A5	Security Misconfiguration	Safe
A6	Vulnerable and Outdated Components	Safe
A7	Identification and Authentication Failures	Unsafe
A8	Software and Data Integrity Failures	Safe
A9	Security Logging and Monitoring Failures	Unsafe
A10	Server-Side Request Forgery (SSRF)	Safe

5. Vulnerability Summary for Unified State Scholarship Portal (grouped by Risk Level)

Note: Only **HIGH** risk vulnerabilities are mentioned here.

#	ISSUE	SEVERITY	IMPACT
1	Privilege Escalation	**HIGH**	It can lead to unauthorized access to sensitive data and functionalities, allowing attackers to impersonate other users and perform actions beyond their intended permissions.
2	SQL Injection	**HIGH**	Attack may result in the unauthorized viewing of user lists, the deletion of entire tables and, in certain cases, the attacker gaining administrative rights to a database, all of which are highly detrimental to a business.
3	Improper Session Implementation	**HIGH**	Attackers can access sensitive data of the application or perform actions that should only be available to authenticated and authorized users.

6. Recommendations

We recommend Tamil Nadu Geographical Information System (TNGIS) - TNeGA to consider the following:

1. Remediate vulnerability (V1-V17) and with more focus on (V1-V3).
2. After fixing the vulnerabilities, inform and ask CYBERNERDS to verify that the vulnerabilities are fully addressed.
3. Minimize application security risks long-term by organizing training in secure application development for developers.
4. A source code review of the application could also be conducted to find even more potential vulnerabilities.

7. Detailed Report

Vulnerability 1

Privilege Escalation

SEVERITY:	CLASSIFICATIONS:	CVSS:
HIGH	CWE-269 *(Improper Privilege Management)* OWASP A04:2021	8.8 *(CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)*

DESCRIPTION

Privilege escalation through response manipulation occurs when an attacker alters a captured request's parameters, such as user IDs, to gain unauthorized access to another user's account.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_backend/api/v1/validate_user

Parameter: user_id
Note: This issue must be fixed across the application.

IMPACT

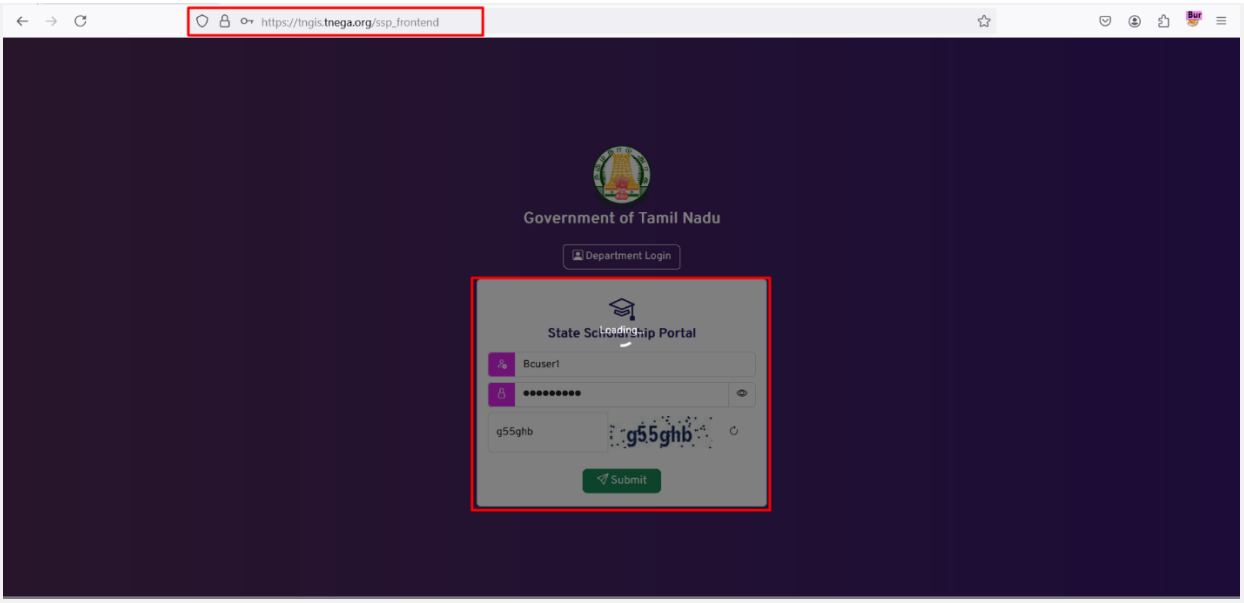
It can lead to unauthorized access to sensitive data and functionalities, allowing attackers to impersonate other users and perform actions beyond their intended permissions.

SOLUTION/WORKAROUND

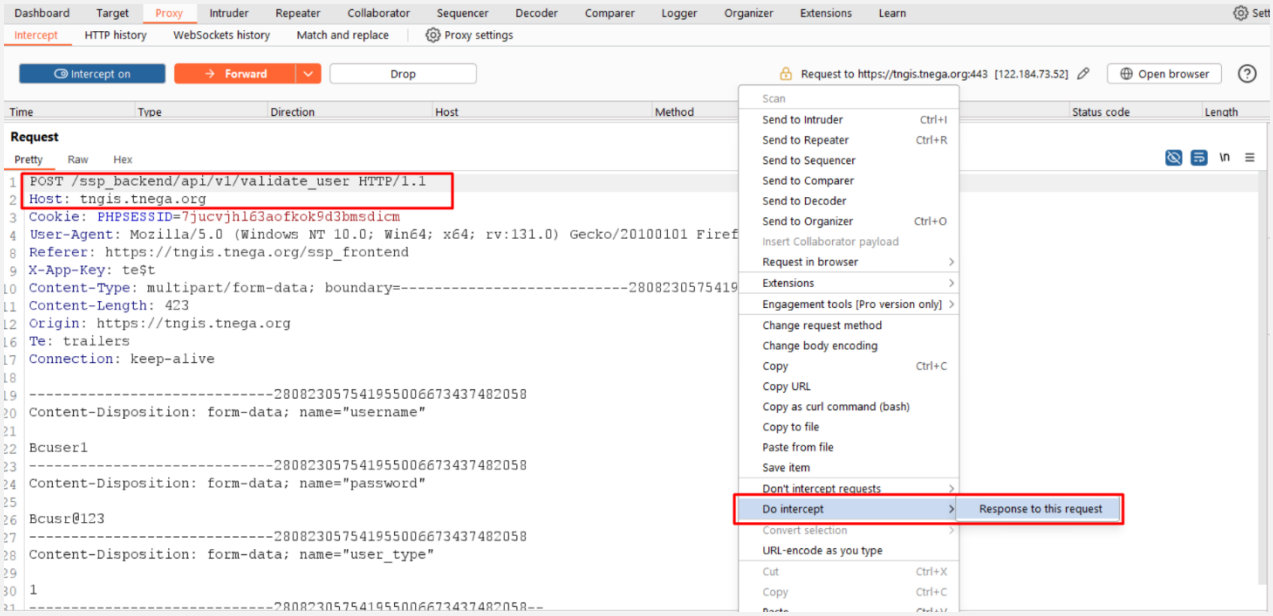
We recommend implementing strict access controls that validate user permissions on the server side for all requests,
Ref: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

STEPS TO REPRODUCE/POC

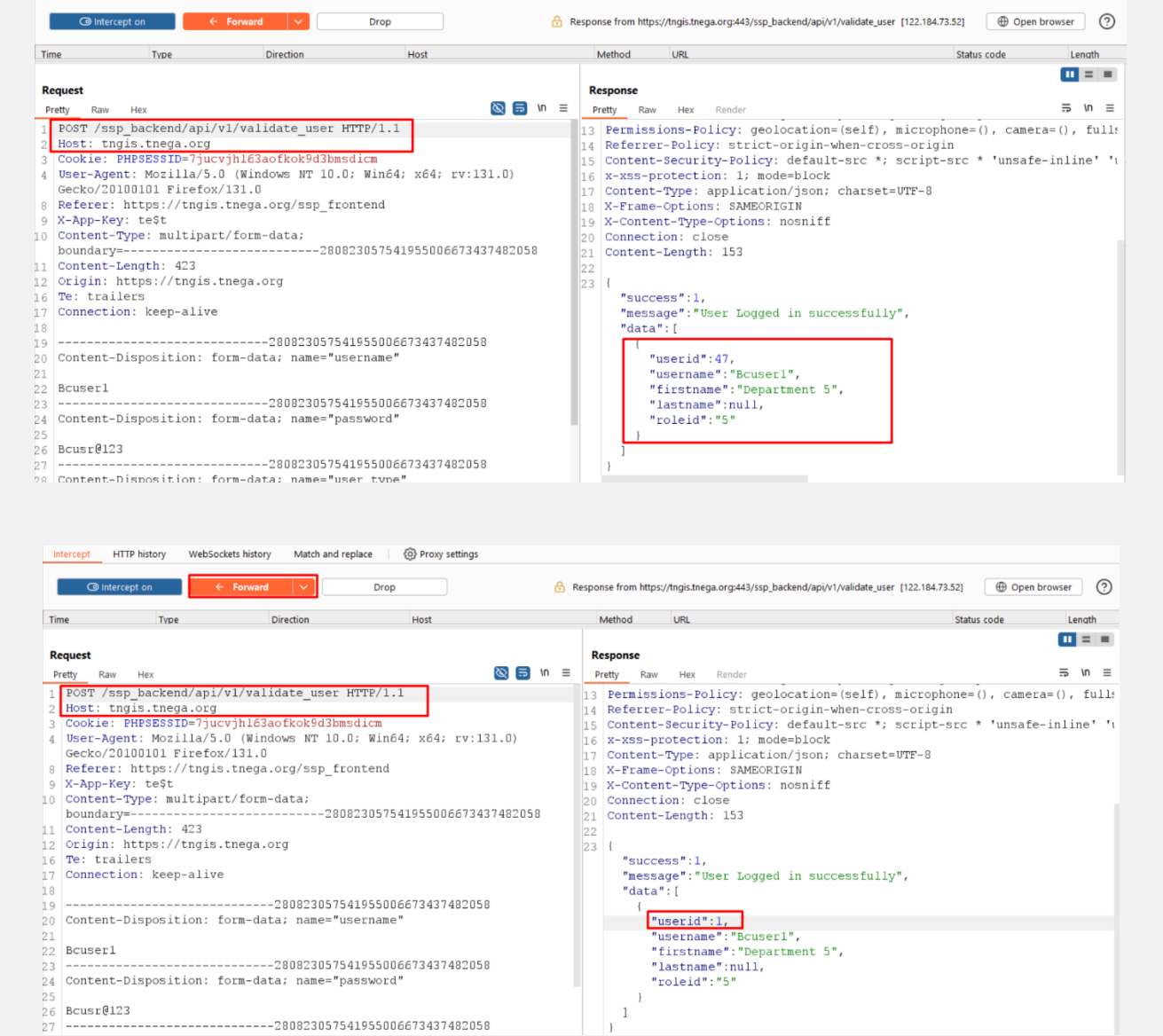
Step 1: Login as “Bcuser1” and capture that request in “Burp Suite”.



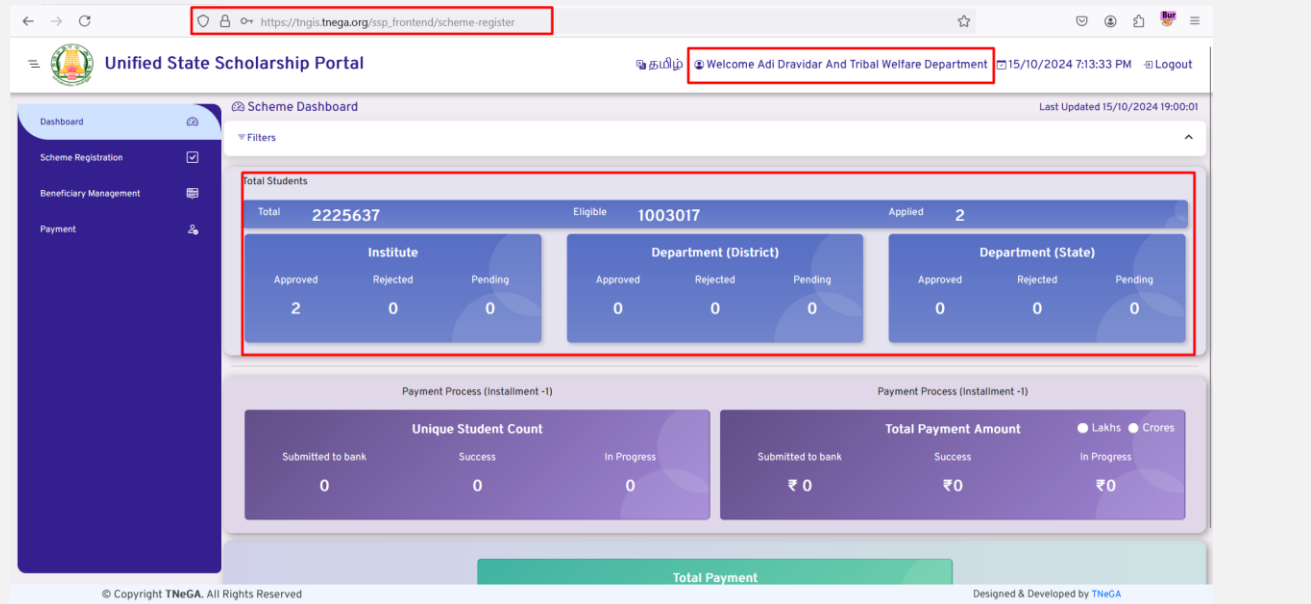
Step 2: Click "Do intercept – Response to this request" for the above captured request and click "Forward"



Step 3: Observe the user_id value associated with "Bcuser1" in the respose. Change the user_id=47 to user_id=1, click "Forward" and turn off the intercept.



Step 4: Now logged in as "Adi Dravidar and Welfare Department."



Vulnerability 2

SQL Injection

SEVERITY:	CLASSIFICATIONS:	CVSS:
HIGH	CWE-89 *(Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection'))* OWASP-A03:2021	8.3 *(CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L)*

DESCRIPTION

SQL injection, also known as SQLI, is a common attack vector that uses malicious SQL code for backend database manipulation to access information that was not intended to be displayed.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_backend/api/v1/get_scheme_list

Parameter: user_id
Note: This issue must be fixed across the application.

IMPACT

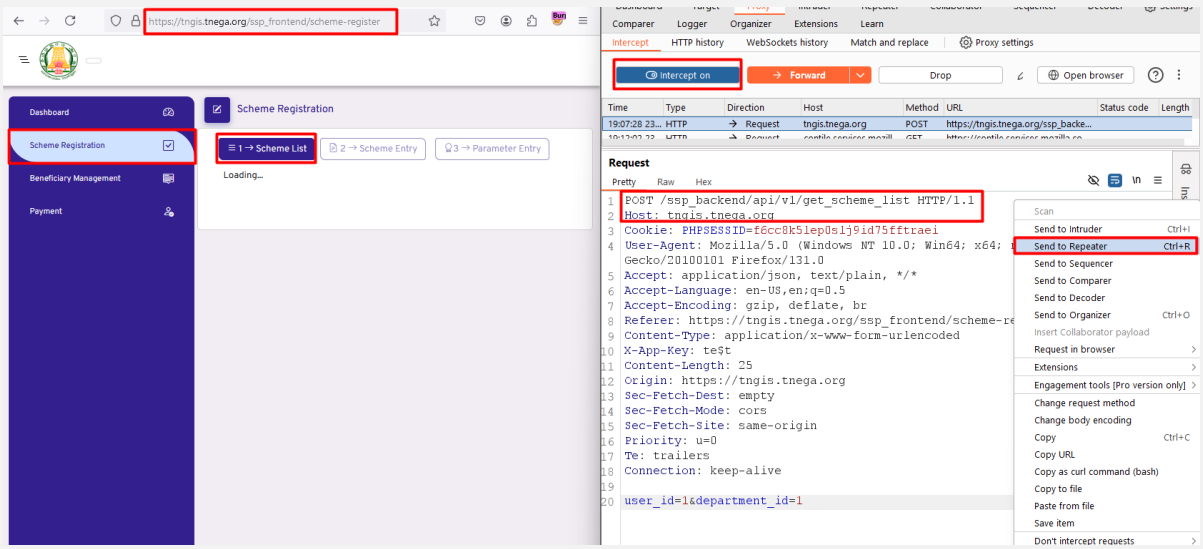
Attack may result in the unauthorized viewing of user lists, the deletion of entire tables and, in certain cases, the attacker gaining administrative rights to a database, all of which are highly detrimental to a business.

SOLUTION/WORKAROUND

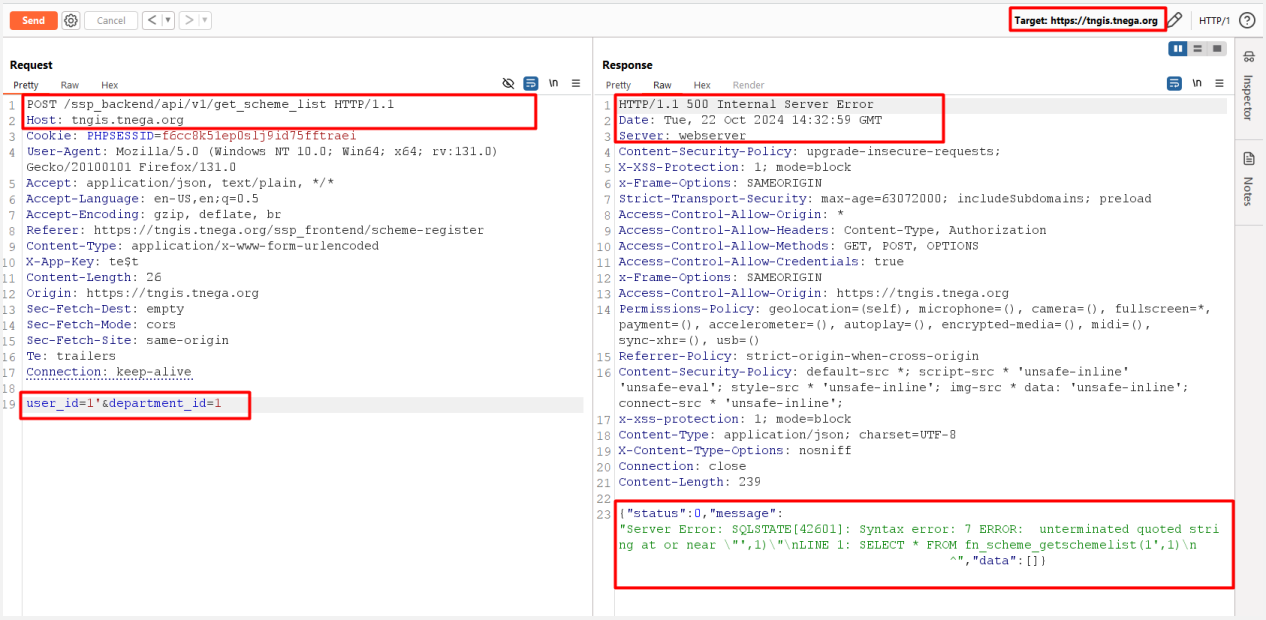
We recommend implementing input validation and parametrized queries including prepared statements. The application code should never use the input directly.

STEPS TO REPRODUCE/POC

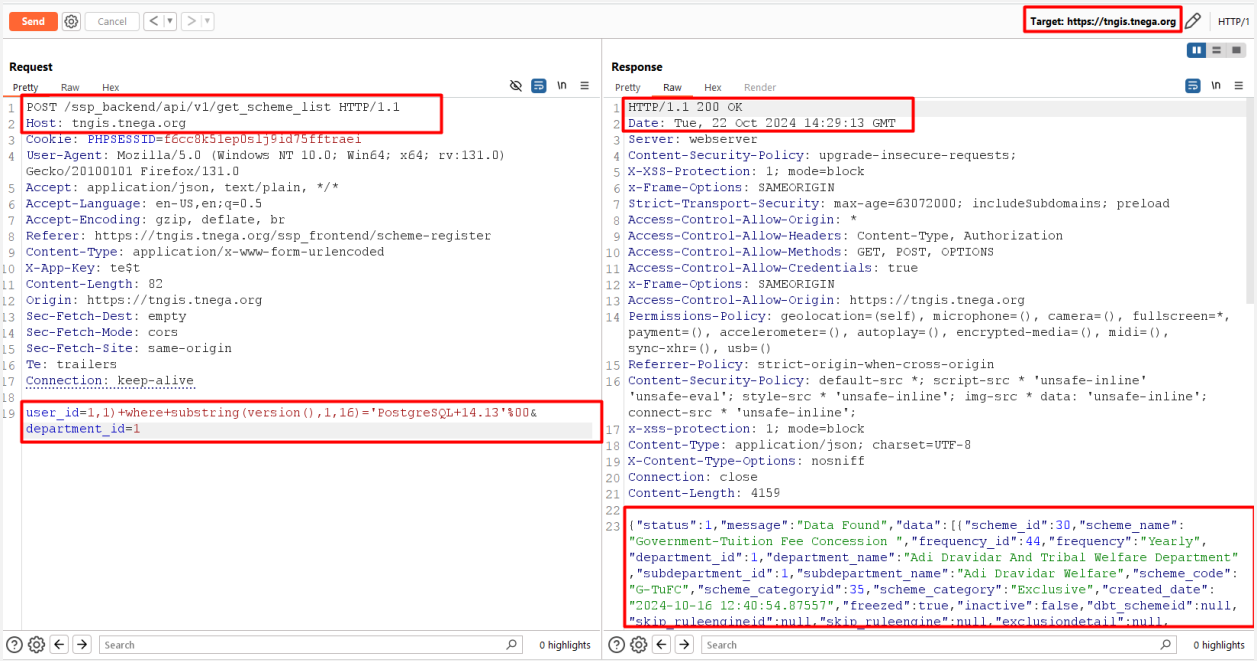
Step 1: Login as "depuser@01". Navigate to "Scheme Registration", click "Scheme List". Capture that request and send it "Repeater"



Step 2: Modify the captured request by inserting a single quote (') in the user_id parameter and click "Send." Observe the response, which display an SQL error.



Step 3: The exploitation can be done furthered, by injecting a payload such as "substring(version(),1,16)="PostgreSQL+14.13'%00", which allows for the identification of the database type along with its version.



Vulnerability 3

Improper Session Implementation

SEVERITY:	CLASSIFICATIONS:	CVSS:
HIGH	CWE-1018 *(Manage user sessions)* OWASP-A07:2021	7.1 *(CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N)*

DESCRIPTION

A web application that allows access to certain resources or functionalities without requiring a valid session or authentication permits unauthorized requests to those resources without users needing to log in or provide valid credentials. This often happens due to misconfigurations or vulnerabilities in the application's access control mechanisms.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/scheme-register

IMPACT

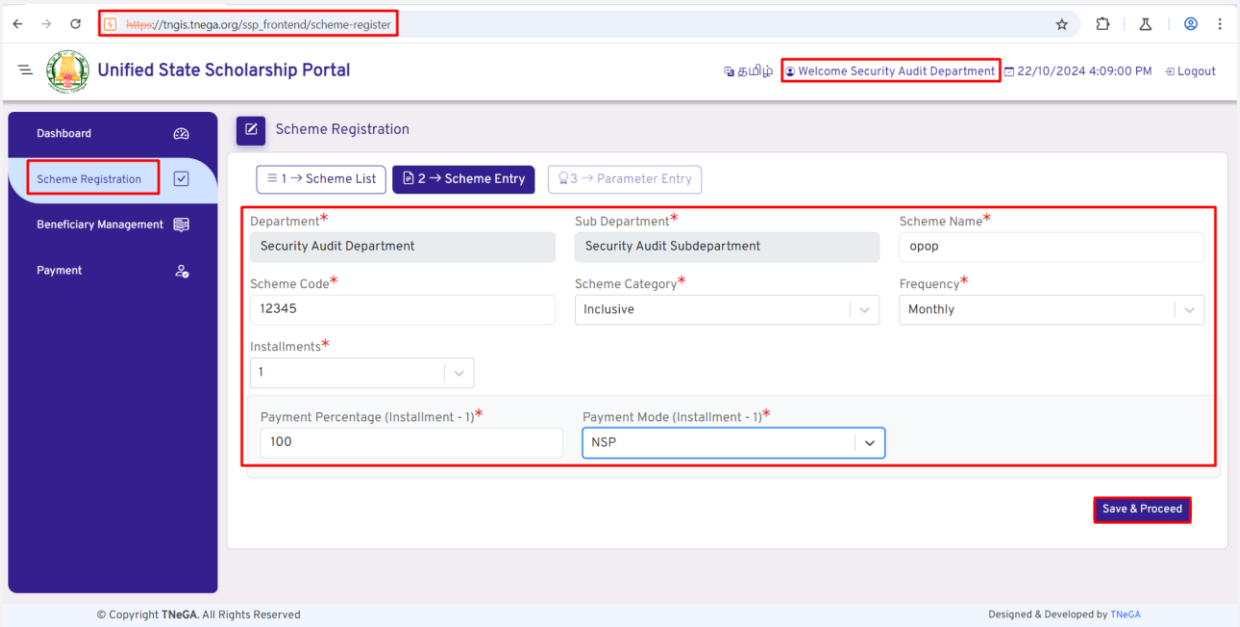
Attackers can access sensitive data of the application or perform actions that should only be available to authenticated and authorized users.

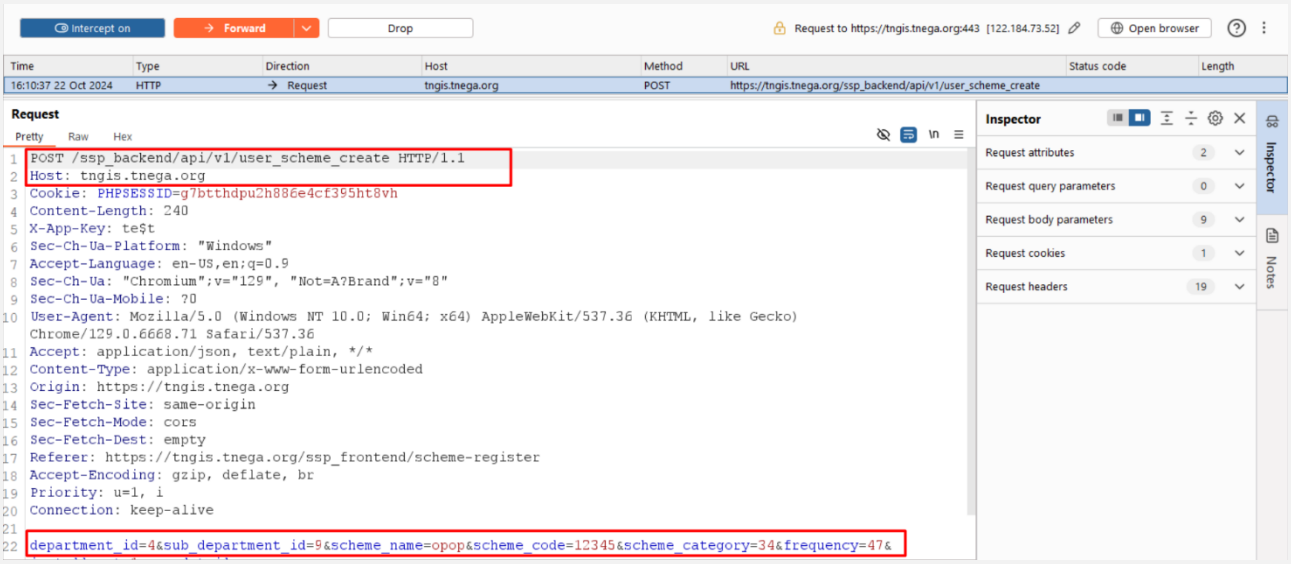
SOLUTION/WORKAROUND

We recommend implementing proper access controls and authentication mechanisms to protect sensitive resources and data. Use secure session management practices to prevent unauthorized modifications.

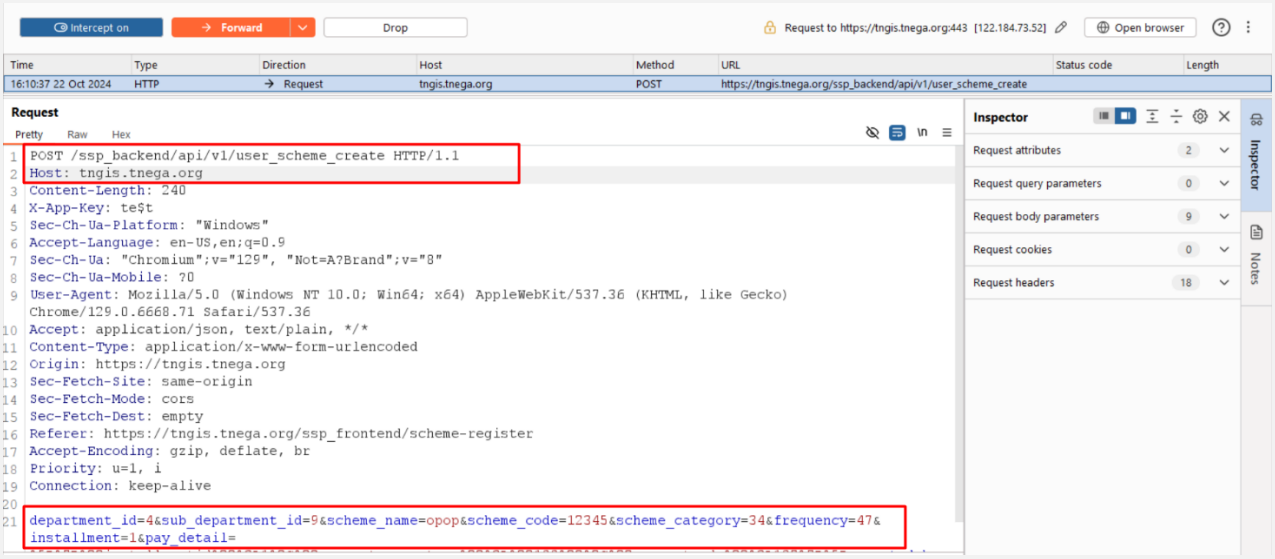
STEPS TO REPRODUCE/POC

Step 1: Login as "testuser@01", navigate to "Scheme Registration", click "Add Scheme" and enter the details. Click "Save and Proceed" and capture that request in "Burp Suite".

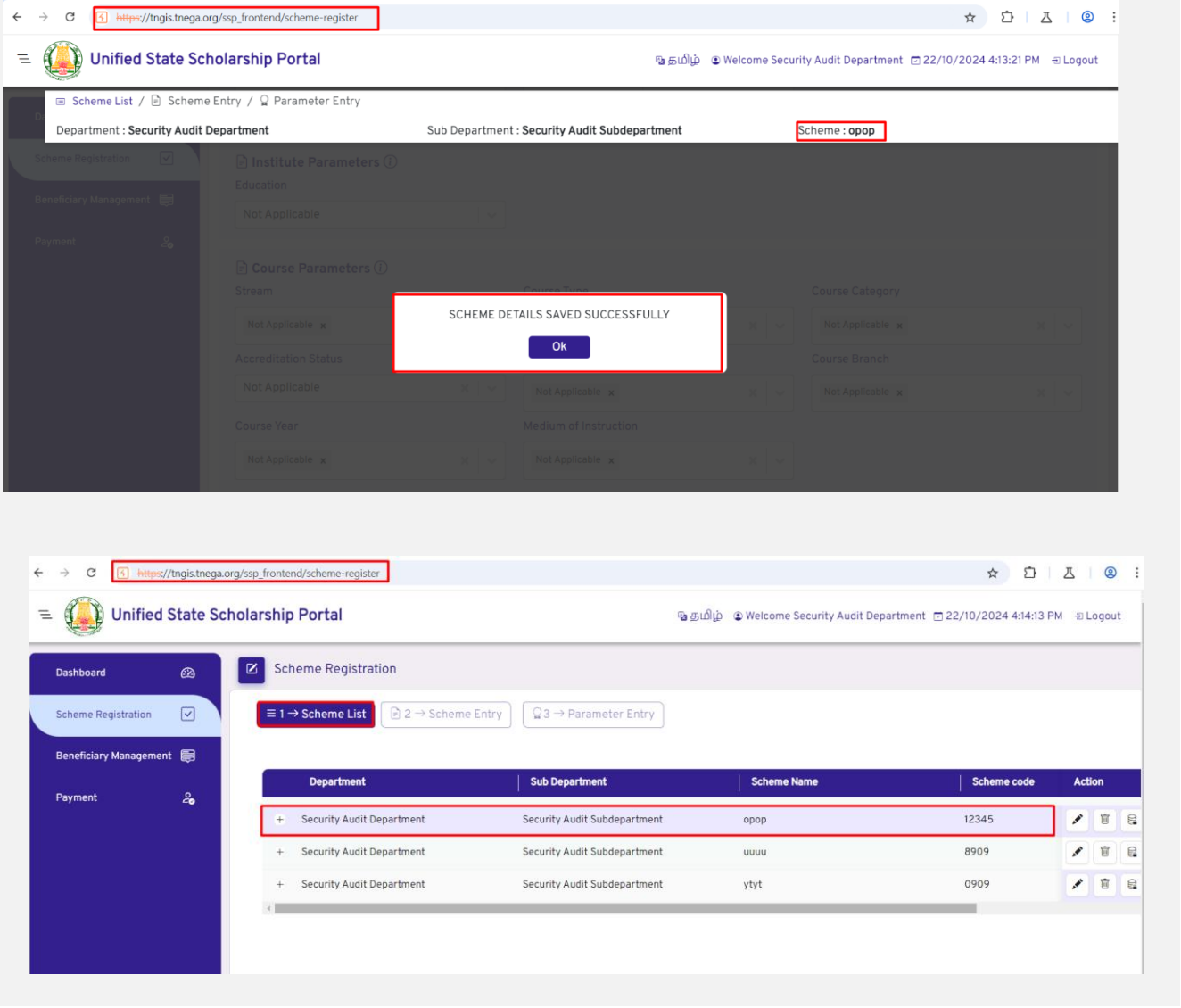




Step 2: Remove the Cookie header from the captured request and click "Forward".



Step 3: It is observed that a new scheme is added successfully, even though there was no valid session cookie.


https://tngis.tnega.org/ssp_frontend/scheme-register
Unified State Scholarship Portal
தமிழ்
Welcome Security Audit Department
22/10/2024 4:13:21 PM
Logout
Scheme List / Scheme Entry / Parameter Entry
Department : Security Audit Department
Sub Department : Security Audit Subdepartment
Scheme : opop
Institute Parameters
Education
Not Applicable
Course Parameters
Stream
Course Category
Accreditation Status
Course Branch
Course Year
Medium of Instruction
SCHEME DETAILS SAVED SUCCESSFULLY
Ok
https://tngis.tnega.org/ssp_frontend/scheme-register
Unified State Scholarship Portal
தமிழ்
Welcome Security Audit Department
22/10/2024 4:14:13 PM
Logout
Dashboard
Scheme Registration
Beneficiary Management
Payment
Scheme Registration
1 → Scheme List
2 → Scheme Entry
3 → Parameter Entry
Department
Sub Department
Scheme Name
Scheme code
Action
Security Audit Department
Security Audit Subdepartment
opop
12345
Security Audit Department
Security Audit Subdepartment
uuuu
8909
Security Audit Department
Security Audit Subdepartment
ytyt
0909

Vulnerability 4

Session Fixation

SEVERITY:	CLASSIFICATIONS:	CVSS:
MEDIUM	CWE-384 *(Session fixation)* OWASP A07:2021	6.3 *(CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:L)*

DESCRIPTION

Session fixation is a type of security vulnerability where an attacker sets or manipulates a user's session identifier before the user authenticates to the server. Once the user logs in with the pre-determined session ID, the attacker can use this ID to gain unauthorized access to the user's session.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

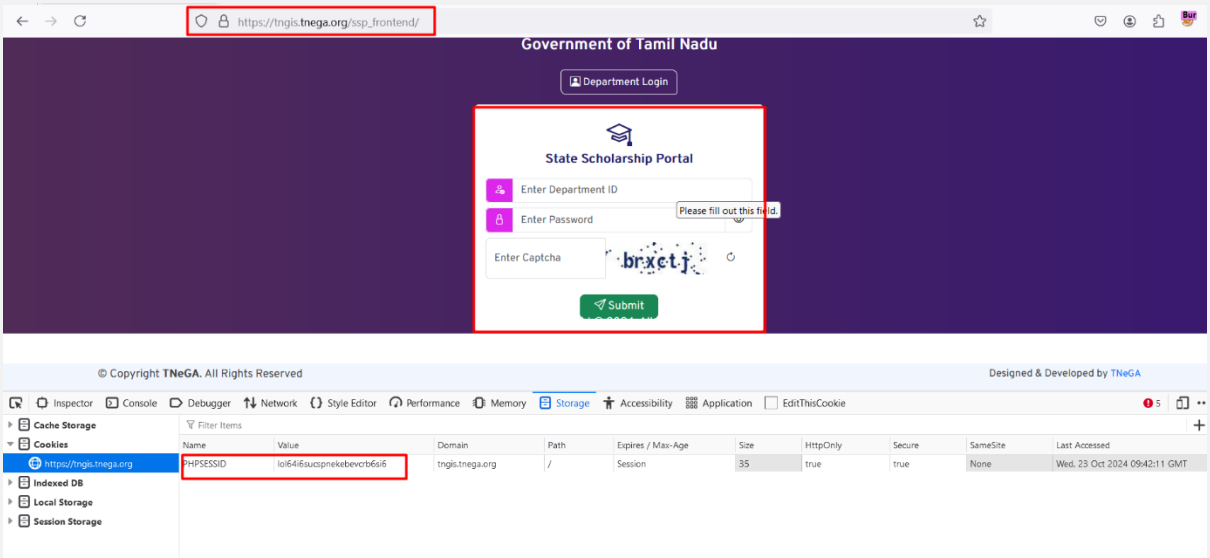
An attacker can force a known session identifier on a user so that, once the user authenticates, the attacker has access to the authenticated session.

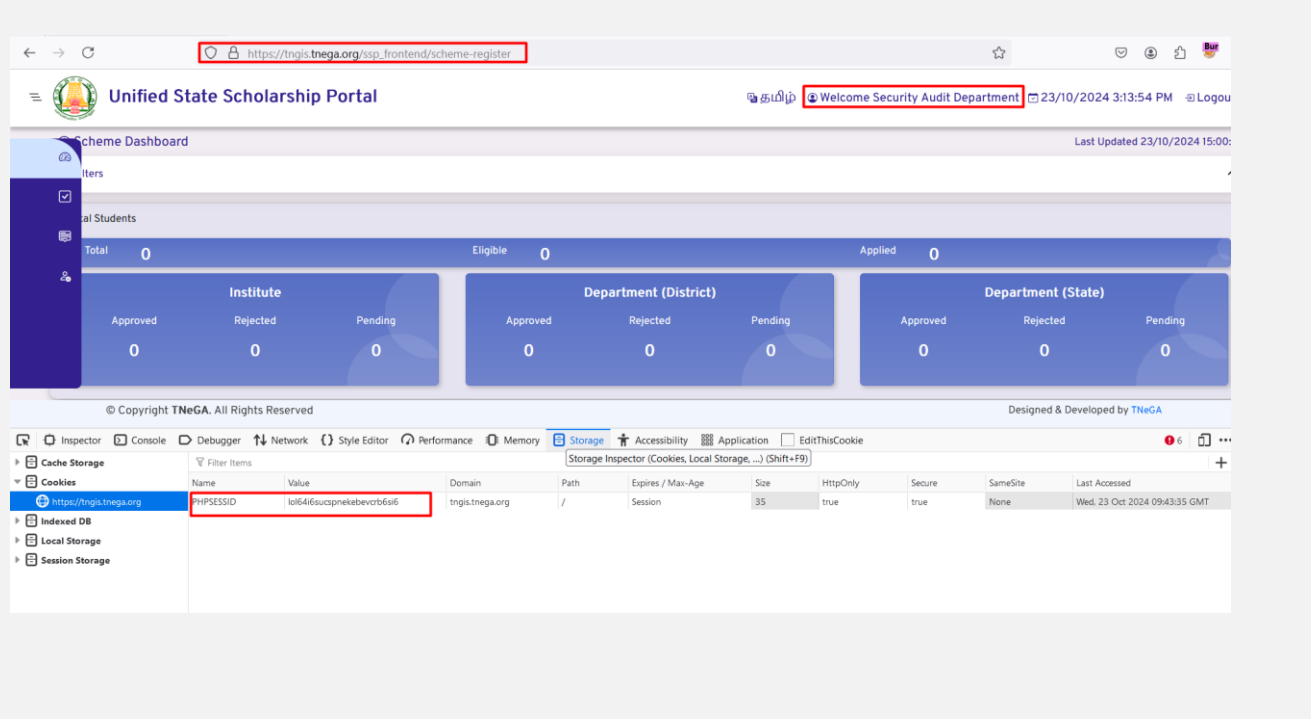
SOLUTION/WORKAROUND

We recommend implementing following safeguards to mitigate this issue:
1. When a user logging into his account the old session ID stored in browser should be expired and new random session ID should be set after successful login of the user.
2. Make sure new session ID is created for every new login of the user.

STEPS TO REPRODUCE/POC

The session id remains the same before and after login of the user, leading to session fixation.




https://tngis.tnega.org/ssp_frontend/scheme-register
Unified State Scholarship Portal
தமிழ்
Welcome Security Audit Department
23/10/2024 3:13:54 PM
Logou
Scheme Dashboard
Last Updated 23/10/2024 15:00:
Total 0
Eligible 0
Applied 0
Institute
Approved 0
Rejected 0
Pending 0
Department (District)
Approved 0
Rejected 0
Pending 0
Department (State)
Approved 0
Rejected 0
Pending 0
© Copyright TNeGA. All Rights Reserved
Designed & Developed by TNeGA
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
EditThisCookie
Storage Inspector (Cookies, Local Storage, ...) (Shift+F9)
Cache Storage
Cookies
https://tngis.tnega.org
Indexed DB
Local Storage
Session Storage
Filter Items
Name
Value
Domain
Path
Expires / Max-Age
Size
HttpOnly
Secure
SameSite
Last Accessed
PHPSESSID
loli64i6sucspnekebevcrb6si6
tngis.tnega.org
/
Session
35
true
true
None
Wed, 23 Oct 2024 09:43:35 GMT

Vulnerability 5

Possibility Of Brute Force Attack

SEVERITY:	CLASSIFICATIONS:	CVSS:
MEDIUM	**CWE-307** *(Improper Restriction of Excessive Authentication Attempts)* **OWASP A07:2021**	**5.6** *(CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:L)*

DESCRIPTION

It is possible to launch an automated brute force attack on the login page since there is no rate limiting implemented. During our Application Audit of the site, we observed that:

1. No 'Account Lockout' policy has been applied for the login page of the user.
2. CAPTCHA not regenerating on failed attempts and not properly validated in the backend.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/
https://tngis.tnega.org/ssp_backend/api/v1/department/paymentApiForReact/fetch_otp_toTransfer_files

IMPACT

An attacker can run brute force attack against the vulnerable application. If such attacks are not handled properly by the application, this can even lead to Denial of Service (DoS) or entire account compromise.

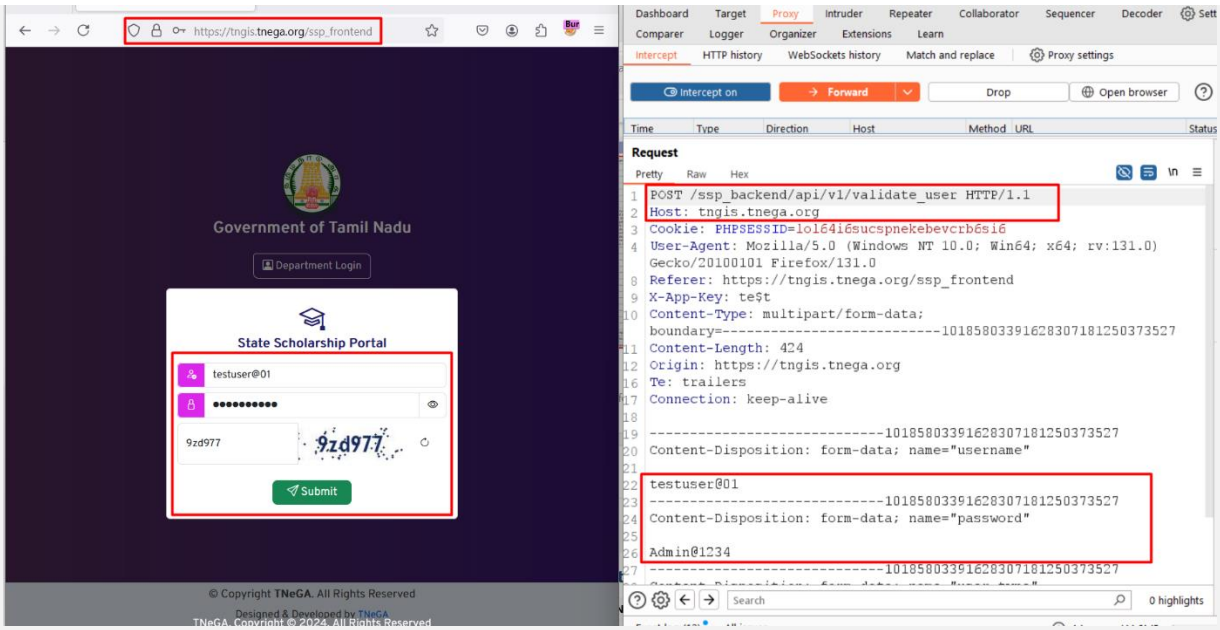
SOLUTION/WORKAROUND

We recommend implementing the following:

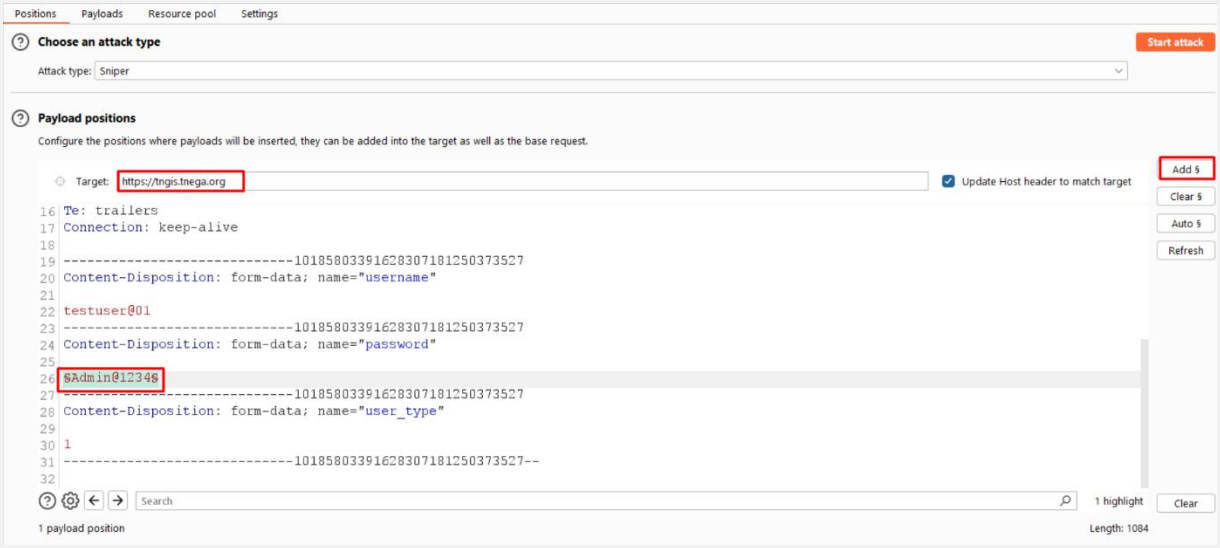
- **Account Lockout:** Lock accounts after a set number of failed attempts (e.g., lock after 5 failed attempts (or) lock for 15 minutes after 5 failed attempts).
- **Rate Limiting:** If the set number of login attempts exceeds defined attempts (e.g., 5 per minute), block the IP address temporarily.
- **CAPTCHA Validation**: Validate the CAPTCHA properly on the backend. Also, regenerate the CAPTCHA after every failed login attempt to ensure that attackers cannot bypass it by reusing the same CAPTCHA.

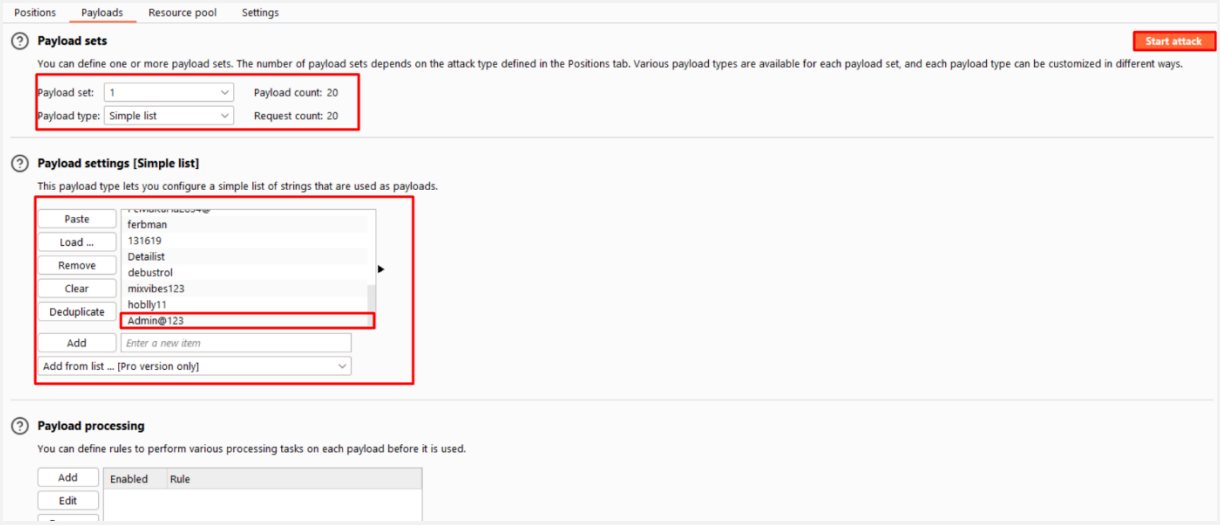
STEPS TO REPRODUCE/POC

Step 1: Login as "testuser@01" and enter an incorrect password. Capture that request in Burp Suite and send it to Intruder.

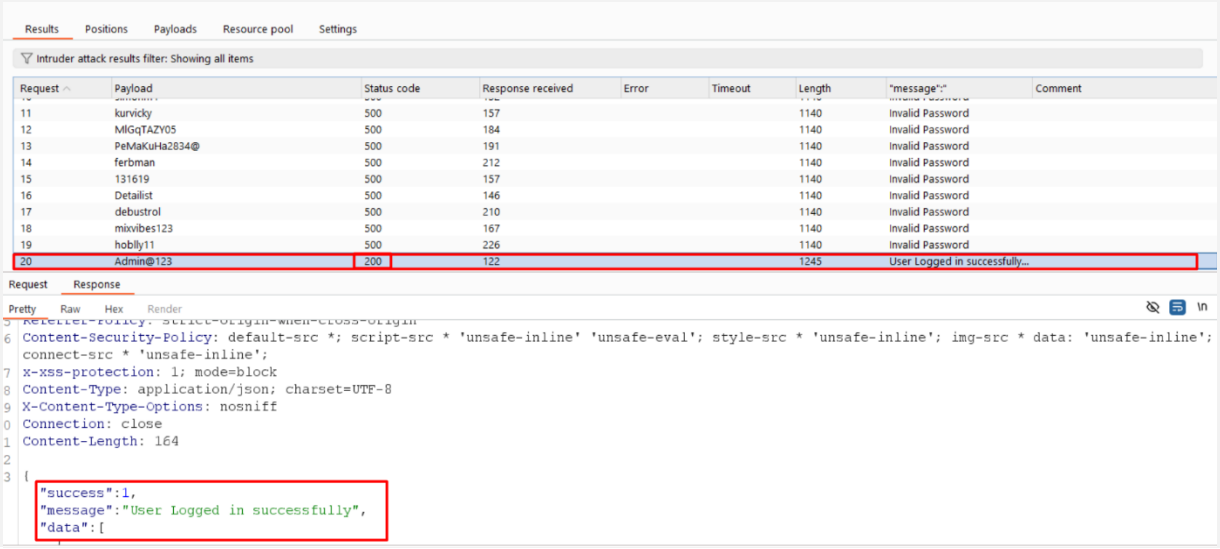


Step 2: Add the "password" parameter to the payloads. Input a list of passwords into the payloads and start the attack.





Step 3: On the 20th attempt, it shows “200 OK” status and “User Logged in successfully”, indicating a successful login.



Vulnerability 6

Insecure Direct Object Reference (IDOR)

SEVERITY:	CLASSIFICATIONS:	CVSS:
MEDIUM	CWE-639 *(Authorization Bypass Through User-Controlled Key)* OWASP A01:2021	4.3 *(CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N)*

DESCRIPTION

The system's authorization functionality does not prevent one user from viewing another department's data or record by modifying the userid value.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_backend/api/v1/get_scheme_list
https://tngis.tnega.org/ssp_backend/api/v1/get_roles

IMPACT

Attackers can exploit IDOR vulnerabilities to access information belonging to another department.

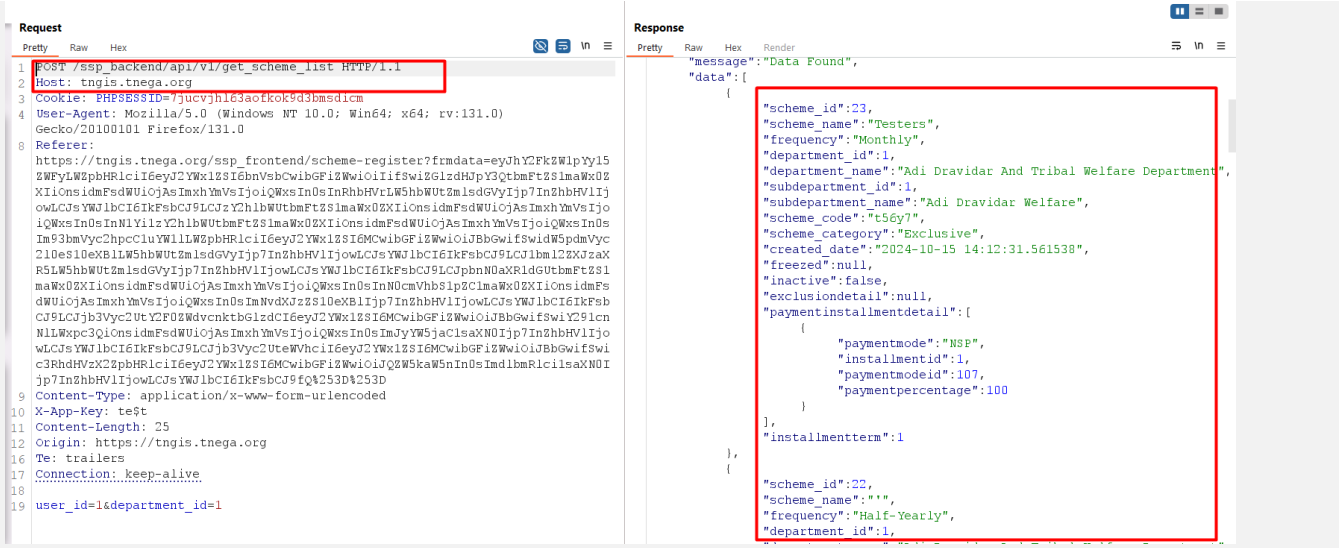
SOLUTION/WORKAROUND

Validate and sanitize user input, to prevent injection attacks and ensure that only legitimate and properly formatted data is accepted by the application.
Enforce access controls at both the application and database levels to restrict users' access to only the data and functionalities they are authorized to use.

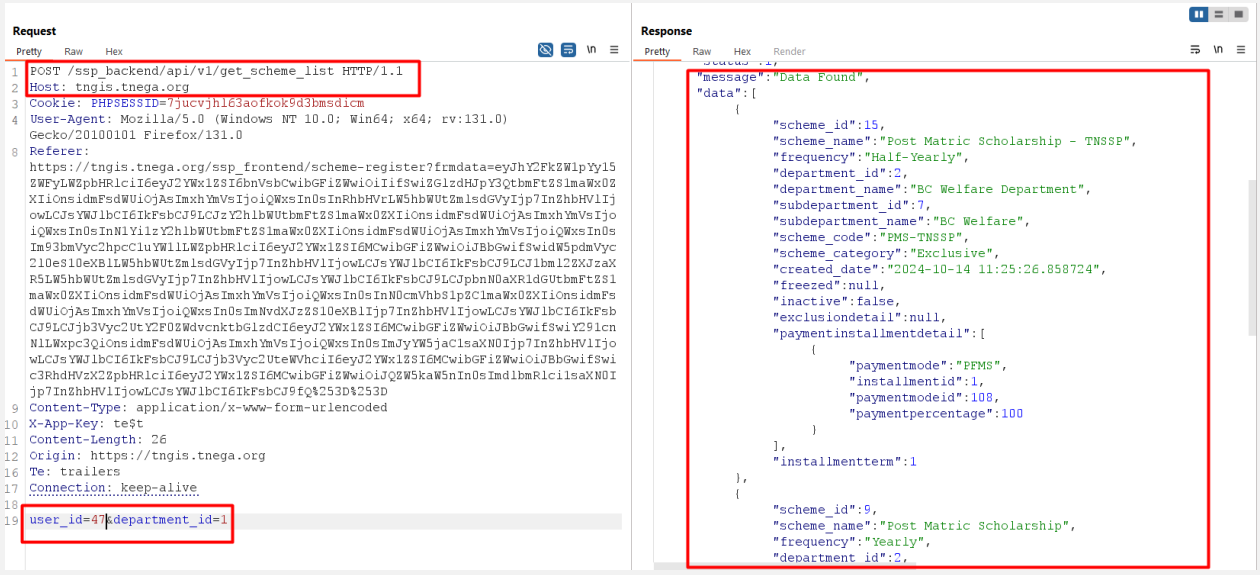
Ref: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

STEPS TO REPRODUCE/POC

Step 1: Login as a "depuser@01", navigate to "Scheme Registration" and capture the request shown below and send it to "Repeater". In the request body, observe that user_id=1 displays "Adi Dravidar And Tribal Welfare Department" informations.



Step 2: By changing user_id=1 to user_id=47, we can view "BC Welfare Department" information.



Vulnerability 7

PHPINFO Public Access

SEVERITY:	CLASSIFICATIONS:	CVSS:
LOW	CWE-200 *(Exposure Of Sensitive Information to An Unauthorized Actor)* OWASP A01:2021	3.7 *(CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N)*

DESCRIPTION

The info page is accessible to public users which discloses information about the current state of PHP's configuration on web server. This includes information about PHP compilation options and extensions, the PHP version, server information and environment (if compiled as a module), the PHP environment, OS version information, paths, master and local values of configuration options, HTTP headers, and the PHP License.

AFFECTED SCOPE

https://tngis.tnega.org/index.php

IMPACT

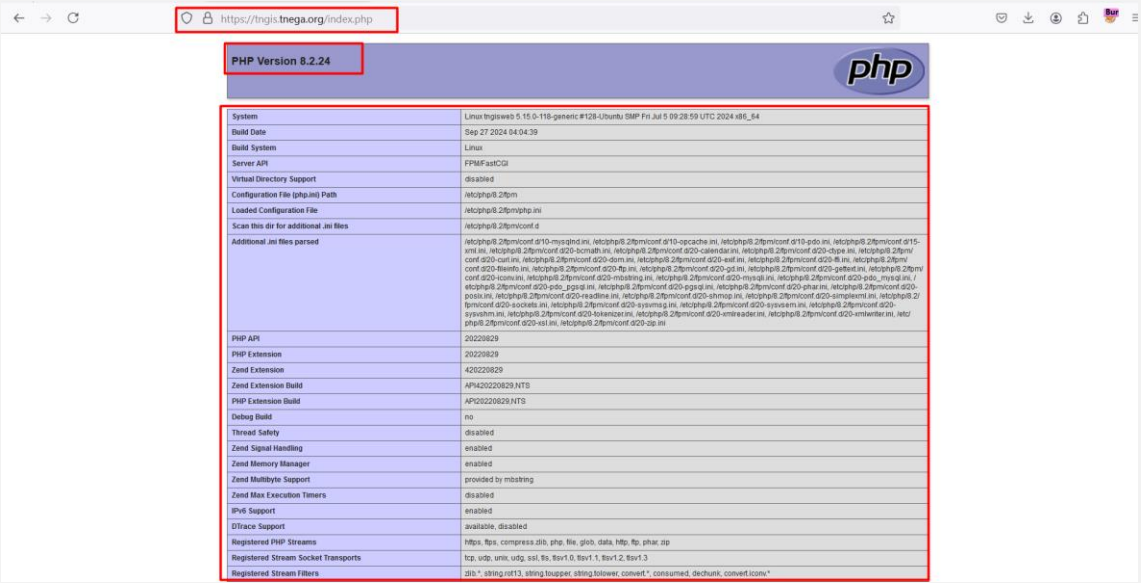
This information can expose server services, OS details, and configuration options, enabling attackers to execute more targeted attacks.

SOLUTION/WORKAROUND

It is recommended to remove the default pages or restrict access from public user.

STEPS TO REPRODUCE/POC

The PHP Info DEFAULT page is accessible publicly as shown below.



Vulnerability 8

Host Header Attack

SEVERITY:	CLASSIFICATIONS:	CVSS:
LOW	CWE-644 *(Improper Neutralization of HTTP Headers for Scripting Syntax)* OWASP A03:2021	3.7 *(CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N)*

DESCRIPTION

Host Header Attack occurs when an attacker manipulates the 'Host' header in an HTTP request to exploit how the server processes the request, which, if not properly validated, could be tampered with by the attacker to perform malicious actions.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_backend/api/v1/department/get_district

NOTE: This issue has to be fixed across the application.

IMPACT

The attacker will be able to redirect and bypass the host header validation which could lead to cache poisoning and compromising the user of the application.

SOLUTION/WORKAROUND

It is recommended to have proper validation for the host header. The request without the whitelisted hostname header should not be accepted.

STEPS TO REPRODUCE/POC

The application does not validate host header value and response with 200 OK status code as shown below.

Request

```
POST /ssp_backend/api/v1/department/get_district HTTP/1.1
Host: evil.com
Cookie: PHPSESSID=7jucvjhl63aofkok9d3bmsdicm
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:131.0)
Gecko/20100101 Firefox/131.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://tngis.tnega.org/ssp_frontend/scheme-register
Content-Type: application/x-www-form-urlencoded
X-App-Key: te$t
Content-Length: 64
Origin: evil.com
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Te: trailers
Connection: keep-alive

academic_year=30&state=0&department=1&sub_department=1&user_id=1
```

Response

```
HTTP/1.1 200 OK
Date: Tue, 15 Oct 2024 07:36:18 GMT
Server: webserver
Content-Security-Policy: upgrade-insecure-requests;
X-XSS-Protection: 1; mode=block
x-Frame-Options: SAMEORIGIN
Strict-Transport-Security: max-age=63072000; includeSubdomains;
preload
Access-Control-Allow-Origin: *
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin: https://tngis.tnega.org
Permissions-Policy: geolocation=(self), microphone=(), camera=(),
fullscreen=*, payment=(), accelerometer=(), autoplay=(),
encrypted-media=(), midi=(), sync-xhr=(), usb=()
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src *; script-src * 'unsafe-inline'
'unsafe-eval'; style-src * 'unsafe-inline'; img-src * data:
'unsafe-inline'; connect-src * 'unsafe-inline';
x-xss-protection: 1; mode=block
Content-Type: application/json; charset=UTF-8
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
```

Vulnerability 9

Inadequate Log Retention

SEVERITY:	CLASSIFICATIONS:	CVSS:
LOW	CWE-778 *(Insufficient Logging)* OWASP A09:2021	3.7 *(CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L)*

DESCRIPTION

It has been observed that the organization's logging system does not retain logs for the required duration of 180 days as recommended by CERT-IN. This lack of proper log retention could hinder post-incident investigations and monitoring.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

If logs are not stored long enough, security teams may miss crucial information, leaving the organization vulnerable to undetected or unresolved threats. This could result in delayed or inadequate response to incidents.

SOLUTION/WORKAROUND

We recommend configuring logging systems to retain logs for a minimum of 180 days, as per CERT-IN guidelines.

STEPS TO REPRODUCE/POC

NA

Vulnerability 10

Insufficient Security logging and Monitoring

SEVERITY:	CLASSIFICATIONS:	CVSS:
LOW	CWE-1355 *(Security Logging And Monitoring Failures)* OWASP A09:2021	3.7 *(CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N)*

DESCRIPTION

Insufficient logging and monitoring refer to the inadequate implementation of logging practices and real-time monitoring capabilities. It occurs when critical events, such as logins, errors, or high-value transactions, are not properly logged or monitored. This oversight can hamper an organization's ability to detect and respond to security incidents promptly, leaving them vulnerable to prolonged attacks and unable to meet compliance requirements.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

Insufficient logging and monitoring refer to the failure to adequately capture and review critical events, which hinders timely detection of security incidents and effective response, potentially exposing sensitive data and failing to meet compliance requirements.

SOLUTION/WORKAROUND

We recommend implementing the following logging, monitoring and process

1. **Suspicious Activities Monitoring**: Continuously monitor for abnormal or unauthorized activities to detect potential security threats promptly.
2. **Remote Logging**: Centralize logging from distributed systems for unified visibility and enhanced security monitoring.
3. **Escalation Process**: Establish a structured procedure to escalate security incidents based on severity to ensure timely responses.
4. **Attack Detection and Protection Measures**: Implement proactive measures like intrusion detection and firewalls to identify and mitigate potential attacks.
5. **Event Logs**: Maintain comprehensive logs of system activities, including user actions and system events, to facilitate auditing and forensic investigations.
6. **Errors and Warning Logs**: Capture and analyse application errors and warnings to proactively address performance issues and security vulnerabilities.

STEPS TO REPRODUCE/POC

NA

Vulnerability 11

Improper Input Validation

SEVERITY:	CLASSIFICATIONS:	CVSS:
LOW	CWE-20 *(Improper Input Validation)* OWASP A03:2021	3.1 *(CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:N)*

DESCRIPTION

It is a weakness that exists when an application does not properly validate or represent input.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/scheme-register

NOTE: This issue must be fixed across the application.

IMPACT

An attacker can inject malicious codes into the application that could lead to SQL injection, Cross site scripting, denial of service, etc. to the application.

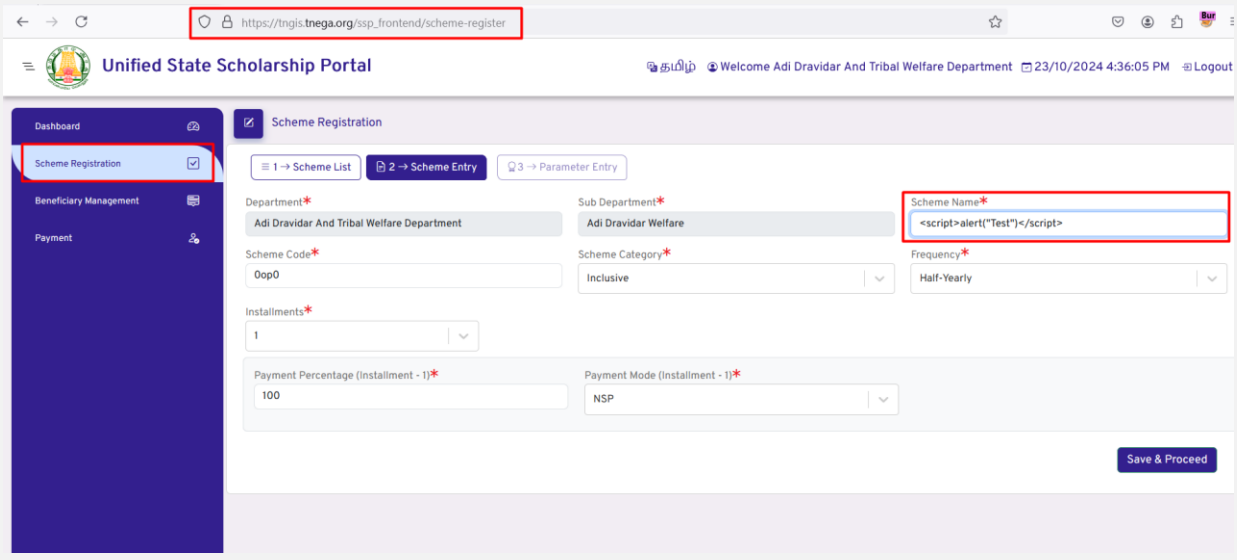
SOLUTION/WORKAROUND

We recommend implementing server-side validation for all user-entered inputs. Only expected values should be accepted. Script tags should be rejected. All user inputs should be sanitized.

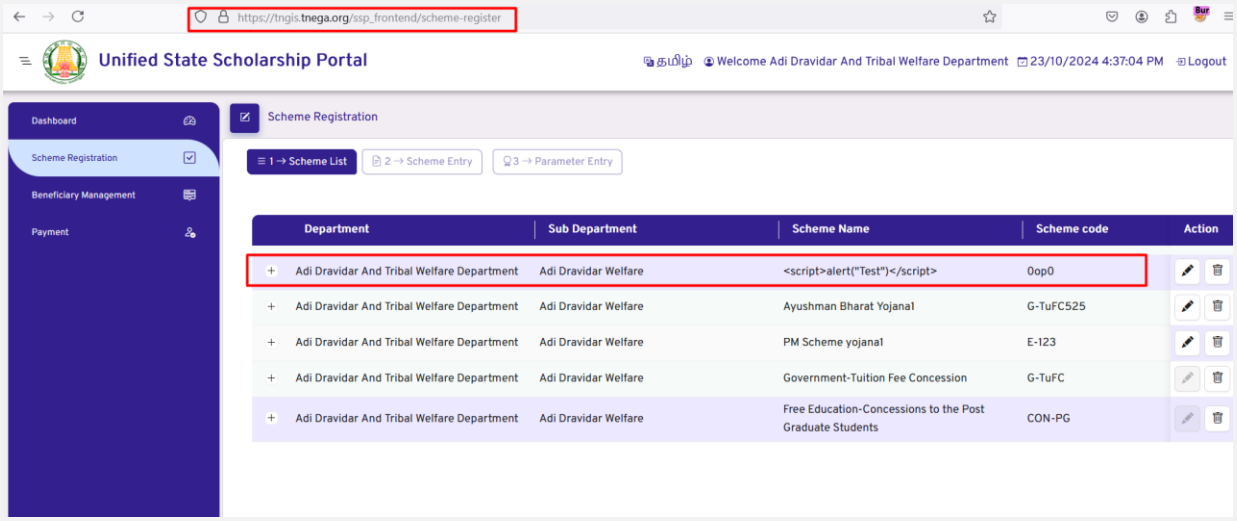
Ref: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

STEPS TO REPRODUCE/POC

Step 1: Login as "depuser@01", navigate to "Scheme Registration", click "Add Scheme, fill in the form, entering "Scheme Name" as <script>alert("Test")</script> and click "Save & Proceed".



Step 2: The scheme was created successfully with given details. It confirms that there is no client side and server-side validation performed.



Vulnerability 12

Stack Trace Error

SEVERITY:	CLASSIFICATIONS:	CVSS:
LOW	CWE-209 *(Generation of Error Message Containing Sensitive Information)* OWASP A04:2021	3.1 *(CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N)*

DESCRIPTION

The application is exposing stack traces, displaying stack traces reveals, detailed information about the file paths, and technologies that should not be exposed to users.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_backend/api/v1/department/paymentApiForReact/file_transfer_python_script

IMPACT

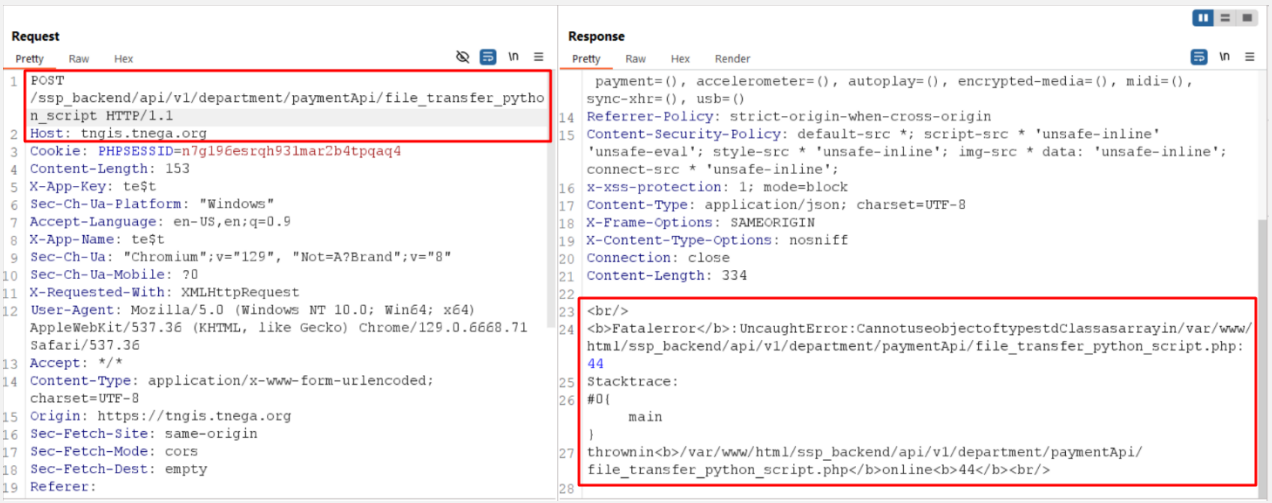
Attackers can use the detailed information from the stack trace to identify weak points in the code, discover software versions with known vulnerabilities, or plan exploits like injection attacks, buffer overflows, etc.

SOLUTION/WORKAROUND

It is recommended to configure the application with custom error pages.

STEPS TO REPRODUCE/POC

By accessing the URL "https://tngis.tnega.org/ssp_backend/api/v1/department/paymentApiForReact/file_transfer_python_script" the application discloses a stack trace error in the response.



Vulnerability 13

Concurrent Login

SEVERITY:	CLASSIFICATIONS:	CVSS:
INFO	CWE-557 *(Concurrency Issues)*	0.0 *(CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N)*

DESCRIPTION

It refers to the situation where a user is logged into a system from multiple devices or sessions simultaneously, potentially posing security risks and inefficiencies.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

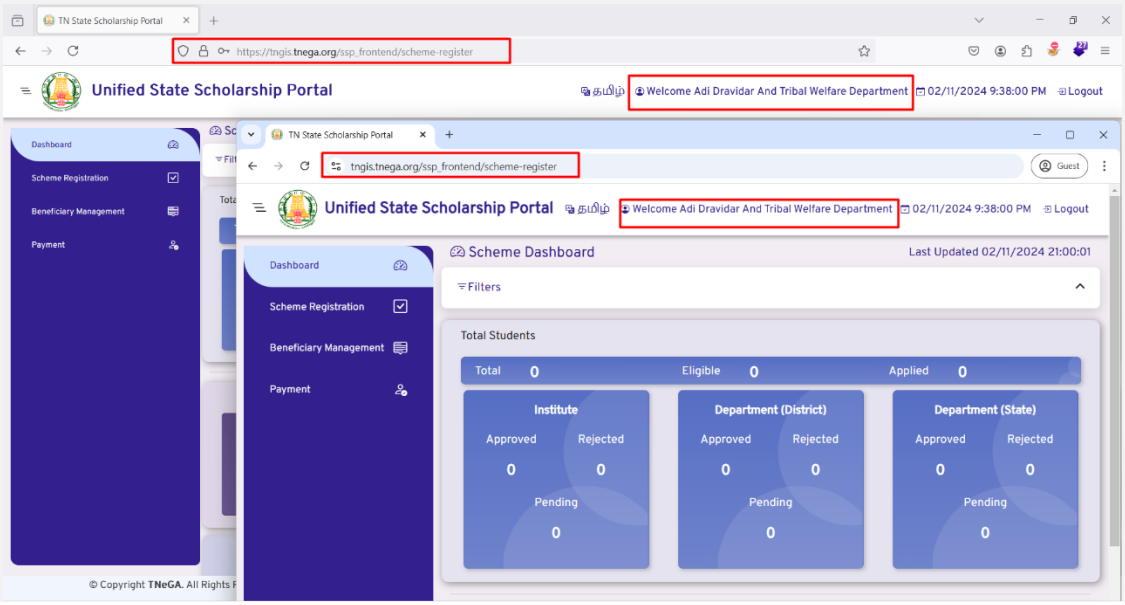
It can lead to unauthorized access, data breaches, and misuse of system resources. They can also result in inconsistent user experiences and data integrity issues.

SOLUTION/WORKAROUND

We recommend implementing session management controls to restrict or manage concurrent logins. This includes enforcing session limits, requiring re-authentication for each new session, and providing users with the ability to manually log out of inactive sessions.

STEPS TO REPRODUCE/POC

The screenshot below shows that the application allows concurrent login.



Vulnerability 14

Vulnerable Remember Password

SEVERITY:	CLASSIFICATIONS:	CVSS:
INFO	CWE-522 *(Insufficiently Protected Credentials)*	0.0 *(CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N)*

DESCRIPTION

It occurs when a website or application allows users to store passwords in their browser or system without proper encryption or security measures. This leaves passwords vulnerable to theft if an attacker gains access to the device or browser.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

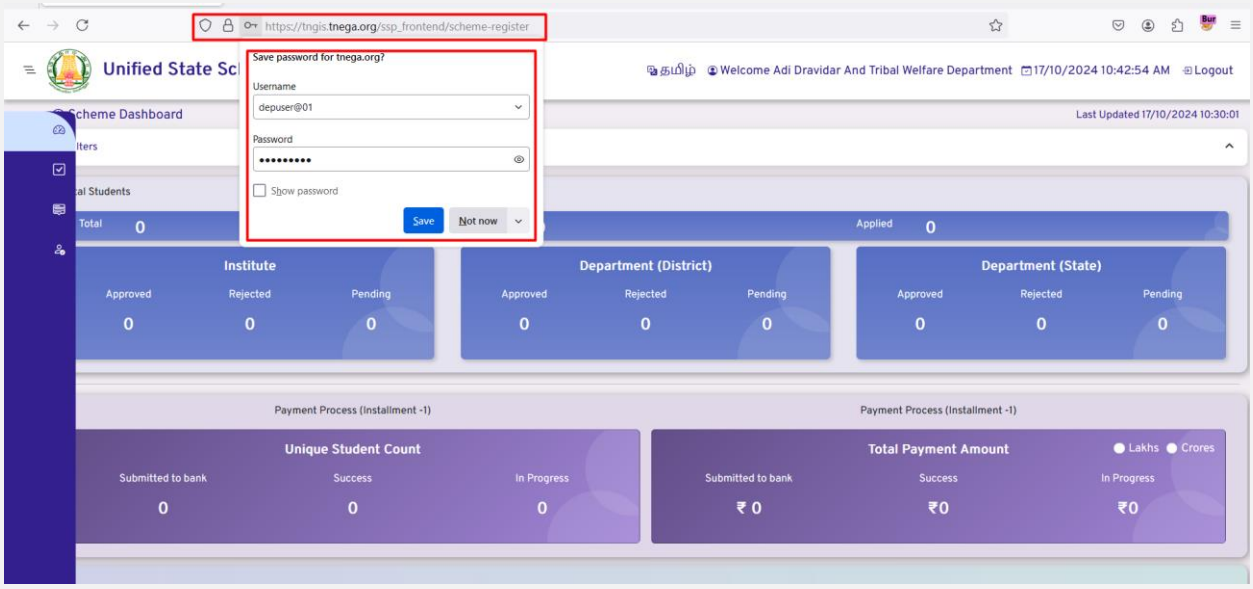
It exposes user credentials to potential theft if an attacker gains access to the user's device or browser. This can lead to unauthorized access to accounts, data breaches, and identity theft.

SOLUTION/WORKAROUND

Ensure that no credentials are stored in clear text or are easily retrievable in encoded or encrypted forms in browser storage mechanisms; they should be stored server-side and follow good password storage practices.

STEPS TO REPRODUCE/POC

The remember password functionality is enabled in the application as shown below.



Vulnerability 15

Change Password Missing

SEVERITY:	CLASSIFICATIONS:	CVSS:
INFO	CWE-640	0.0 *(CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N)*

DESCRIPTION

Process of updating user authentication credentials to enhance security. Typically involves verification of identity and replacing the existing password with a new, stronger one.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

If password changes are not implemented on a website, the risk of unauthorized access increases. Users may continue to utilize weak or compromised passwords, leaving their accounts vulnerable to exploitation and potentially leading to security breaches and data theft.

SOLUTION/WORKAROUND

We recommend implementing a password management system that encourages or enforces regular password changes.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

STEPS TO REPRODUCE/POC

NA

Vulnerability 16

Forgot Password Missing

SEVERITY:	CLASSIFICATIONS:	CVSS:
INFO	CWE-640	0.0 *(CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N)*

DESCRIPTION

To implement a proper user management system, systems integrate a Forgot Password service that allows the user to request a password reset.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

Without "forgot password" on the applicaion, user or admins might struggle to regain access, causing delays in work, increased support requests, and potential security risks due to poor password practices

SOLUTION/WORKAROUND

We recommend implementing a forgot password management system that encourages or enforces regular password changes.

STEPS TO REPRODUCE/POC

NA

Vulnerability 17

2FA Missing

SEVERITY:	CLASSIFICATIONS:	CVSS:
INFO	CWE-308 *(Use Of Single-Factor Authentication)*	0.0 *(CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:N)*

DESCRIPTION

Two-factor authentication (2FA) is a specific type of multi-factor authentication (MFA) that strengthens access security by requiring two methods (also referred to as authentication factors) to verify your identity. These factors can include something you know like a username and password — plus something you have — like a smartphone app — to approve authentication requests.

AFFECTED SCOPE

https://tngis.tnega.org/ssp_frontend/

IMPACT

An attacker may be able to find a legitimate account and password via brute force, reverse brute force, or dictionary attacks.

SOLUTION/WORKAROUND

We recommend proper implementation of 2FA for all admin accounts.

STEPS TO REPRODUCE/POC

NA

8. ANNEXURE – A (List of Test Performed)

S/N	Test(s) Performed	Test Result	S/N	Test(s) Performed	Test Result
1	Test Network Infrastructure Configuration	Safe	2	Test Application Platform Configuration	Unsafe
3	Test File Extensions Handling for Sensitive Information	Unsafe	4	Review Old Backup and Unreferenced Files for Sensitive Information	Safe
5	Enumerate Infrastructure and Application Admin Interfaces	Safe	6	Test HTTP Methods	Safe
7	Test HTTP Strict Transport Security	Safe	8	Test RIA Cross Domain Policy	Safe
9	Test File Permission	Safe	10	Test for Subdomain Takeover	NA
11	Test Cloud Storage	NA	12	Test Role Definitions	NA
13	Test User Registration Process	NA	14	Test Account Provisioning Process	NA
15	Testing for Account Enumeration and Guessable User Account	Safe	16	Testing for Weak or Unenforced Username Policy	Safe
17	Testing for Credentials Transported over an Encrypted Channel	Safe	18	Testing for Default Credentials	Safe
19	Testing for Weak Lock Out Mechanism	Unsafe	20	Testing for Bypassing Authentication Schema	Unsafe
21	Test for Sensitive Data in GET Request	Safe	22	Test for Vulnerable Remember Password	Unsafe
23	Testing for Browser Cache Weakness	Safe	24	Testing for Weak Password Policy	Safe
25	Testing for Weak Security Question Answer	NA	26	Testing for Weak Password Change or Reset Functionalities	NA
27	Testing for Weaker Authentication in Alternative Channel	NA	28	Testing Directory traversal	Safe
29	Testing for Bypassing Authorization Schema	Unsafe	30	Testing for Privilege Escalation	Unsafe
31	Testing for Insecure Direct Object References	Unsafe	32	Testing for Session Management Schema	Unsafe
33	Testing for Session Fixation	Unsafe	34	Testing for Cookies attributes	Safe
35	Testing for Exposed Session Variables	Safe	36	Testing for Cross Site Request Forgery	Safe
37	Testing for Logout Functionality	Unsafe	38	Testing Session Timeout	Unsafe
39	Testing for Session Puzzling	Unsafe	40	Testing for Session Hijacking	Safe
41	Testing for Reflected Cross Site Scripting	Safe	42	Testing for Stored Cross Site Scripting	Safe
43	Testing for HTTP Parameter Pollution	Safe	44	Testing for SQL Injection	Unsafe
45	Testing for Code Injection	Safe	46	Testing for Command Injection	Safe
47	Testing for Incubated Vulnerability	Safe	48	Testing for HTTP Splitting/Smuggling	Safe

49	Testing for HTTP Incoming Requests	Safe	**50**	Testing for Host Header Injection	Unsafe
51	Testing for Server-side Template Injection	Safe	**52**	Testing for Server-Side Request Forgery	Safe
53	Testing for Improper Error Handling	Unsafe	**54**	Testing for Weak Transport Layer Security	Safe
55	Testing for Padding Oracle	NA	**56**	Testing for Sensitive information sent via unencrypted channels	Safe
57	Testing for Weak Encryption	Safe	**58**	Test Business Logic Data Validation	Safe
59	Test Ability to Forge Requests	Safe	**60**	Test Integrity Checks	Safe
61	Test for Process Timing	NA	**62**	Test Number of Times a Function Can Be Used Limits	Unsafe
63	Testing for the Circumvention of Workflows	NA	**64**	Test Defenses Against Application Misuse	Safe
65	Test Upload of Unexpected File Types	NA	**66**	Test Upload of Malicious Files	NA
67	Testing for DOM based Cross Site Scripting	Safe	**68**	Testing for JavaScript Execution	Safe
69	Testing for HTML Injection	Safe	**70**	Testing for Client-side URL Redirect	Safe
71	Testing for CSS Injection	Safe	**72**	Testing for Client-side Resource Manipulation	Safe
73	Testing Cross Origin Resource Sharing	Safe	**74**	Testing for Cross Site Flashing	Safe
75	Testing for Clickjacking	Safe	**76**	Testing WebSockets	NA
77	Testing Web Messaging	NA	**78**	Testing Browser Storage	Safe
79	Testing for Cross Site Script Inclusion	Safe	**80**	Testing GraphQL	Safe

9. ANNEXURE - B (Methodology)

During the assessment, the following international best practices and methodologies are used:

- Open Web Application Security Frameworks Project Testing Guide v4 (OWASP)
- Institute for Security and Open Methodologies – Open-Source Security Testing Methodology Manual 3 (OSSTMM)
- Open Information Systems Security Group – Information Systems Security Assessment Framework 0.2.1 (ISSAF)

The experience and certifications of the experts involved in the methodologies applied ensure that the quality of our assessments always meet the highest expectations and demands.

A security assessment provides organizations with a clear picture of the likely business disruption/damage in the event of a malicious hacker attacking critical IT resources of the organization. Some of the highlights of the approach are as follows:

- Minimal / No disruption to business
- Comprehensive and deep investigation
- Flexible, location-neutral approach
- Effective reporting
- Remediation support
- Re-Test

10. ANNEXURE – C (Glossary)

Hacker
A word was given by the media to define what we will more accurately call attacker intruder or cracker.

Vulnerability
A bug in a system that may be abused to gain privileges.

Exploit
A program or strategy to exploit vulnerability.

Identity
Identity is who someone or what something is, for example, the name by which something is known.

Authentication
Authentication is the process of confirming the correctness of the claimed identity.

Authorization
Authorization is the approval, permission, or empowerment for someone or something to do something.

Denial of Service
The prevention of authorized access to a system resource or the delaying of system operations and functions

Hardening
Hardening is the process of identifying and fixing vulnerabilities in a system.

Risk
Risk is the product of the level of threat with the level of vulnerability. It establishes the likelihood of a successful attack.

Threat
A potential for violation of security, which exists when there is a circumstance, capability, action, or event that could breach security and cause harm.

Confidentiality
Confidentiality is the need to ensure that information is disclosed only to those who are authorized to view it.

Integrity
Integrity is the need to ensure that information has not been changed accidentally or deliberately and that it is accurate and complete.

Availability
Availability is the need to ensure that the business purpose of the system can be met and that it is accessible to those who need to use it.

11. ANNEXURE - D (Test Types)

WHITE-BOX

The penetration test team has complete carte blanche access to the target and has been supplied with network diagrams, hardware, operating system, and application details, etc., and before a test being carried out. This does not equate to a truly blind test but can speed up the process a great deal and leads to more accurate results being obtained. The amount of prior knowledge leads to a test targeting specific operating systems, applications, and network devices that reside on the network rather than spending time enumerating what could be on the network. This type of test equates to a situation whereby an attacker may have complete knowledge of the internal network.

BLACK-BOX

No prior knowledge of a company network is known. In essence, an example of this is when an external web-based test is to be carried out and only the details of a website URL or IP address is supplied to the testing team. It would be their role to attempt to break into the company website/ network. This would equate to an external attack carried out by a malicious hacker.

GREY-BOX

The testing team would simulate an attack that could be carried out by a disgruntled, disaffected staff member. The testing team would be supplied with appropriate user-level privileges and a user account and access permitted to the internal network by relaxation of specific security policies present on the network i.e., port level security.

12. ANNEXURE – E (Application Vulnerabilities)

Injection
Injection flaws, such as SQL, OS, and LDAP injection, occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing unauthorized data.

Cross-Site Scripting (XSS)
XSS flaws occur whenever an application takes untrusted data and sends it to a web browser without proper validation and escaping. XSS allows attackers to execute scripts in the victim's browser which can hijack user sessions, deface websites, or redirect the user to malicious sites.

Broken Authentication and Session Management
Application functions related to authentication and session management are often not implemented correctly, allowing attackers to compromise passwords, keys, session tokens, or exploit other implementation flaws to assume other users' identities.

Insecure Direct Object References
A direct object reference occurs when a developer exposes a reference to an internal implementation object, such as a file, directory, or database key. Without an access control check or other protection, attackers can manipulate these references to access unauthorized data.

Cross-Site Request Forgery
A CSRF attack forces a logged-on victim's browser to send a forged HTTP request, including the victim's session cookie and any other automatically included authentication information, to a vulnerable web application. This allows the attacker to force the victim's browser to generate requests the vulnerable application thinks are legitimate requests from the victim.

Security Misconfiguration
Good security requires having a secure configuration defined and deployed for the application, frameworks, application server, web server, database server, and platform. All these settings should be defined, implemented, and maintained as many are not shipped with secure defaults. This includes keeping all software up to date, including all code libraries used by the application.

Insecure Cryptographic Storage
Many web applications do not properly protect sensitive data, such as credit cards, SSNs, and authentication credentials, with appropriate encryption or hashing. Attackers may steal or modify such weakly protected data to conduct identity theft, credit card fraud, or other crimes.

Failure to Restrict URL Access
Many web applications check URL access rights before rendering protected links and buttons. However, applications need to perform similar access control checks each time these pages are accessed, or attackers will be able to forge URLs to access these hidden pages anyway.

Insufficient Transport Layer Protection
Applications frequently fail to authenticate, encrypt, and protect the confidentiality and integrity of sensitive network traffic. When they do, they sometimes support weak algorithms, use expired or invalid certificates, or do not use them correctly.

Unvalidated Redirects and Forwards
Web applications frequently redirect and forward users to other pages and websites and use untrusted data to determine the destination pages. Without proper validation, attackers can redirect victims to phishing or malware sites or use forwards to access unauthorized pages.

Web-Based client Attacks

Phishing is the criminally fraudulent process of attempting to acquire sensitive information such as usernames, passwords, and credit card details by masquerading as a trustworthy entity in an electronic communication.

Cross-Site Tracing

Cross-Site Tracing (XST) attack involves the use of XSS and the HTTP TRACE function. HTTP TRACE is a default function in many web servers, primarily used for debugging. The client sends an HTTP TRACE with all header information including cookies, and the server simply responds with that same data. If using JavaScript or other methods to steal a cookie or other information is disabled through the use of an HttpOnly cookie or otherwise, an attacker may force the browser to send an HTTP TRACE request and send the server response to another site. HttpOnly is an extra parameter added to cookies which hide the cookie from the script (supported in most, but not all browsers.